LAMP: ASP Rule Induction and Optimization with LLMs

David Bunker
University of Potsdam

Abstract

Answer Set Programming (ASP) offers a compelling mechanism for knowledge representation and reasoning, however developing such programs remains challenging. This work introduces the LLM to ASP Modeling Protocol (LAMP), a system by which an LLM iteratively generates and refines candidate ASP rules with each candidate verified by the Clingo solver. LAMP is evaluated against the leading Inductive Logic Programming (ILP) system Inductive Learning of Answer Set Programs (ILASP), across 120 synthetic benchmarks of varying complexity. ILASP achieves 97.5% accuracy, with failures due to search space timeouts. Among LLMs, *gpt-5-mini* reaches 100% accuracy *gpt-oss:20b* 95.8%, and *qwen3-coder:30b* 45.0%. Although ILASP was on average faster, the leading LLM solved all benchmarks on which ILASP timed out. Both LAMP and ILASP are also able to reduce literal counts and increase stratification at a similar level. Finally, feature analysis is used to determine predictors of LLM and ILASP success or failure.

1 Introduction

Knowledge representation and reasoning problems arise across many domains, including software and hardware customization, industrial manufacturing, service composition and others. These problems are characterized by combinatorial constraints that determine which combinations of components are valid. Answer Set Programming (ASP) models such problems, offering clear semantics with expressive capabilities for constraints, defaults, and combinatorial reasoning.

However, writing ASP programs that are both correct and performant remains challenging, as modeling choices can significantly affect solver behavior. Recent progress in Large Language Models (LLMs) offers new opportunities to reduce this modeling burden. LLMs have shown strong capabilities in code generation and program synthesis, suggesting they may assist in deriving ASP encodings from high level natural language specifications [14].

It has also been shown LLMs can perform inductive logic programming, such as learning logic rules from background knowledge and positive and negative examples when combined with feedback from a formal inference engine, such as Prolog [10]. Here the authors proposed a systematic evaluation framework graded by expressivity to quantify LLM strengths and limitations in logic theory induction.

The LLM to ASP Modeling Protocol (LAMP) is designed to guide LLMs in deducing and optimizing ASP rules from given instance facts and expected answer sets. LAMP provides a structured interaction protocol that enables refinement of ASP encodings, with the goal of improving both correctness and performance. Beyond proposing the system itself, LAMP can be used as a framework for evaluating the capabilities of different LLMs in ASP program generation.

Both LAMP and the classical Inductive Logic Programming (ILP) system Inductive Learning of Answer Set Programs (ILASP) [18] address the same underlying problem class. Given a set of instance facts and an expected answer set, find a program within a bounded hypothesis space that produces the correct answer sets. ILASP solves this via exhaustive search under the Learning from Answer Sets (LAS) framework while LAMP replaces exhaustive search with LLM guided generation and Clingo based iterative verification. As the expected input and output are the same a controlled comparison can be made between symbolic ILP search and LLM based generation on the same class of tasks.

The relevant research questions of this work are the following.

RQ1. Is a structured approach for LLM assisted ASP modeling possible using the LAMP system and if so under what conditions do LLMs outperform, match, or fall short of classical ILP ILASP on ASP rule induction?

RQ2. Does this system allow for comparative evaluation of LLMs and ILASP on ASP generation and optimization tasks?

RQ3. Which ASP language features most strongly affect LAMP and ILASP performance?

This paper investigates the current capabilities of Large Language Models (LLMs) regarding Answer Set Programming (ASP) and offers directions for future research at the intersection of artificial intelligence and constraint solving. The LAMP system can be compared against similar systems such as those based on LLMs fine-tuned to ASP [6], iterative scheduling and plan generation with LLMs and ASP [20], and LLM based systems that can learn directly from ASP answer sets [4].

2 Background

This section presents the core background underpinning LAMP including Answer Set Programming (ASP), Inductive Learning of Answer Set Programs (ILASP), and Large Language Models (LLMs).

2.1 Answer Set Programming

ASP is a declarative programming paradigm based on stable model semantics [3].

Syntactic Primitives. A *term* is either a constant (a lowercase symbol or integer such as a, 0), a variable (an uppercase symbol, such as X), or a function symbol $f(t_1, \ldots, t_k)$ applied to terms. A *predicate* p/k is a symbol of arity $k \geq 0$. Applying it to k terms yields an *atom* $p(t_1, \ldots, t_k)$. A ground atom contains no variables. A *literal* is either a positive literal a (an atom) or a *default-negated* literal *not* a, where *not* denotes negation as failure and *not* a holds whenever a cannot be derived.

Rules and Programs. An ASP program Π is a finite set of *rules* of the following form.

$$h \leftarrow b_1, \ldots, b_m, not\ c_1, \ldots, not\ c_n \tag{1}$$

Here h is the *head* (a classical atom or empty), each b_i is a positive body literal, and *not* c_j denotes default negation of atom c_j. When $m = n = 0$ the rule is a *fact* and when h is empty the rule is a *constraint* (written $\leftarrow$ body), eliminating any answer set in which the body is satisfied.

Variables and Grounding. Rules are written with first order variables, allowing compact expression over large domains. Prior to solving, a *grounder*, such as Gringo, substitutes all domain consistent constants for each variable, producing an equivalent propositional ground program Π_g. A rule is *safe* if every variable in the head or in a negative body literal also appears in a positive body literal, which guarantees that grounding is finite and well defined.

Semantics. Let $\mathcal{A}$ denote the Herbrand base set of all ground atoms. Given a ground program Π and a set $S \subseteq \mathcal{A}$, the *Gelfond–Lifschitz reduct* Π^S is obtained by the steps.

1. Remove every rule whose body contains *not* c for some $c \in S$
2. Remove all negative literals from the bodies of remaining rules.

S is a *stable model*, referred to in ASP as an *answer set*, of Π if and only if S is the minimal model of the reduct Π^S. A program may have zero, one, or multiple answer sets and solutions to the encoded problem correspond exactly to its answer sets.

Modern ASP solvers, such as Clingo [12], extend the core language with aggregates, choice rules, disjunctive heads, and optimization statements. LAMP uses Clingo as its verification oracle but operates over a restricted subset of normal rules only, as defined in Section 3.1.

Solving Complexity. Deciding whether a ground ASP program has an answer set is Σ_2^P-complete [2]. This places ASP one level above NP in the polynomial hierarchy, reflecting the two sources of computational hardness, the nondeterministic search over candidate sets $S \subseteq \mathcal{A}$, and the co-NP check that no proper subset of S is also a stable model of Π^S.

2.2 Learning from Answer Sets

Inductive Learning of Answer Set Programs (ILASP) [18] is a framework for Inductive Logic Programming (ILP) that learns ASP programs from examples. ILASP operates over three inputs. These are a *hypothesis space* $\mathcal{H}$, a *background knowledge* program B, and a set of *learning examples* E.

Hypothesis Space. The hypothesis space $\mathcal{H}$ is defined by a *mode bias*, which constrains the structure of rules that may appear in a learned hypothesis. A *mode declaration* is either a *head declaration* $modeh(a)$ or a *body declaration* $modeb(a)$, specifying which atoms may appear in rule heads and bodies respectively. Each atom in a mode declaration may contain *placeholder terms* of the form $\#var(t)$ (a variable of type t) or $\#con(t)$ (a constant of type t), which are instantiated during the search. A *hypothesis* $H \in \mathcal{H}$ is a set of ASP rules consistent with the mode bias.

Learning Examples. ILASP supports three types of examples, each placing a different constraint on the answer sets of $B \cup H$:

- **Positive examples** $e^+ \in E^+$: at least one answer set of $B \cup H$ must satisfy e^+.
- **Negative examples** $e^- \in E^-$: no answer set of $B \cup H$ may satisfy e^-.
- **Context-dependent examples** $e^{cd} \in E^{cd}$: a pair $\langle e_c, e_p \rangle$ where e_c is a *context* (an additional ASP program) and e_p is a *partial interpretation*. At least one answer set of $B \cup H \cup e_c$ must extend e_p, meaning it must include all atoms in e_p^+ and exclude all atoms in e_p^-.

Context-dependent examples subsume positive and negative examples and are the primary example type in modern ILASP, enabling the expression of partial observations under specific conditions.

Learning Task. A *learning task* is a tuple $T = \langle B, \mathcal{H}, E^+, E^-, E^{cd} \rangle$. A hypothesis $H \in \mathcal{H}$ is an *inductive solution* of T if it covers all examples. Every positive and context-dependent example is satisfied by some answer set of $B \cup H \cup e_c$, and no answer set of $B \cup H$ satisfies any negative example. When hypotheses are assigned a *cost* (typically the sum of rule lengths), the learning task seeks an *optimal* solution. This is the inductive solution of minimum cost.

Search Procedure. ILASP reduces the learning task to a sequence of ASP solving problems. The hypothesis space is encoded as an ASP metaprogram, and the search for a covering hypothesis is directed by a *conflict-driven* procedure [18]. Candidate hypotheses are proposed, checked against all examples, and conflicts, examples not yet covered, are used to prune the space and guide subsequent iterations. This conflict-driven approach, implemented in the FastLAS system, scales substantially better than exhaustive enumeration of $\mathcal{H}$.

Solving Complexity. Learning an optimal inductive solution under the full ILASP framework is Σ_3^P-complete [1], one level above the Σ_2^P-hardness of ASP itself. The additional level reflects the search over hypotheses $H \in \mathcal{H}$ layered on top of the Σ_2^P check that H is a valid inductive solution. This theoretical hardness motivates an alternative approach, rather than exhaustive symbolic search, LAMP replaces the hypothesis search with LLM-guided generation, bypassing the Σ_3^P search.

2.3 Large Language Models for Code Generation

Large Language Models (LLMs) are neural language models trained on large amounts of text and source code, enabling strong performance on code generation tasks. Given a structured prompt, an LLM generates a candidate program token by token. Its output is non-deterministic and may require post-processing or validation against a formal oracle.

LLMs have been shown to generate logic programs from natural language specifications [14] and to perform inductive logic programming when paired with a formal inference engine [10]. A key pattern across these systems is *iterative refinement*: the LLM generates a candidate, the candidate is verified symbolically, and any mismatch is encoded as feedback for the next prompt. LAMP instantiates this pattern for ASP, using Clingo as the verification oracle and Clingo-derived feedback to guide successive generation attempts.

3 Methodology

The LAMP framework operates on a formally defined benchmark structure that constrains the hypothesis space and the target answer set, as illustrated

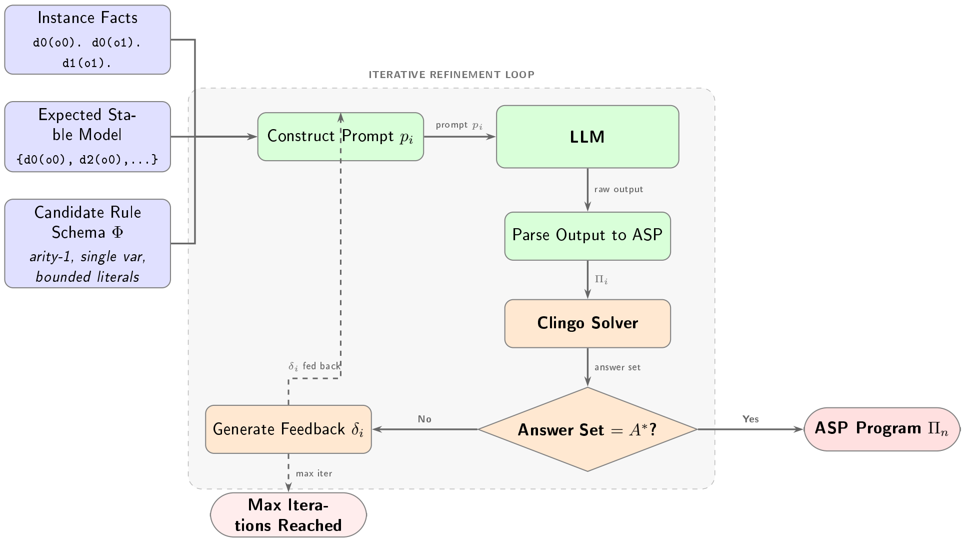

Figure 1: Overview of the LAMP system architecture.

in Figure 1. This section defines that benchmark framework, describes how benchmark instances are generated, and then details the LAMP interaction protocol and evaluation criteria.

3.1 Benchmark Framework

To keep the benchmarks tractable for both LLM generation and ILP search, the ASP used here is deliberately restricted. Rather than a full configuration system such as Object-Oriented ASP (OOASP) [9], programs consist only of unary *facts* of the form `descriptor(object)` and *normal rules* of the form `new_descriptor(X) :- descriptor_0(X), [not] descriptor_1(X), ...` which derive new object descriptions from initial instance facts. This subset is expressive enough to require fairly complex rule induction while remaining within a reasonable hypothesis space.

The following is an example representing the model facts that objects `"car"` and `"scooter"` are both vehicles, represented with the predicate `vehicle`, and the scooter is slow moving, represented with the predicate `slow_moving`. It also has the rule that if an object is a vehicle and not slow moving it is highway possible, represented with the predicate `highway_possible`.

Listing 1: Vehicle example ASP program.

```
vehicle("scooter").
vehicle("car").
slow_moving("scooter").
highway_possible(X) :- vehicle(X), not slow_moving(X).
```

Running this results in the answer set `{ vehicle("scooter"), vehicle("car"), slow_moving("scooter"), highway_possible("car") }`. Given these facts and rules, `highway_possible(car)` can be deduced, indicating the car is highway possible and, due to default negation, the scooter is not highway possible.

This restricted ASP subset can now be formalized. A *Complexity Category* specifies the size and shape of the problem of how many predicates and terms exist, how many facts are given, and how large the rules may be. Together with the input facts and a target answer set, it defines a concrete benchmark that both LAMP and ILASP can solve.

Previous work on LLM based logic rule induction has characterized complexity by structural rule graph topology such as Chain, Rooted Directed Graph, Disjunctive, and Mixed categories [10]. Since this work is concerned with correct answer set deduction and rule optimization rather than rule graph structure, complexity is instead quantified by six numerical features. These include number of possible predicates, terms, facts, rules, positive body literals per rule, and negative body literals per rule. All generated programs are also limited to having exactly one answer set.

Definition 1 (Complexity Category)**.** *A* Complexity Category *is a tuple*

$$\mathcal{C} \;=\; \langle\, \mathcal{D},\; \mathcal{T},\; N_F,\; R,\; L^+,\; L^-,\; A_{\min} \,\rangle$$

where:

- $\mathcal{D}$ *is a finite set of* descriptors *(predicate symbols);*
- $\mathcal{T}$ *is a finite set of* objects *(constant symbols);*
- $N_F \in \mathbb{N}$ *is the number of* instance facts*;*
- $R \in \mathbb{N}$ *is the maximum number of* rules*;*
- $L^+ \in \mathbb{N}$ *is the maximum number of* positive literals *in each rule body;*
- $L^- \in \mathbb{N}$ *is the maximum number of* negative literals *in each rule body; and*
- $A_{\min} \in \mathbb{N}$ *is the* minimum derived atoms*, the minimum number of atoms that must appear in the answer set beyond the given input facts.*

From a Complexity Category we can define a concrete input instance. Each instance pairs a set of ground facts and a target answer set with a bounded schema that defines the hypothesis space both LAMP and ILASP search over.

Definition 2 (Input Instance). *Given a Complexity Category stated as* $\mathcal{C} = \langle \mathcal{D}, \mathcal{T}, N_F, R, L^+, L^-, A_{\min} \rangle$ *(Definition 1), an* instance *of* $\mathcal{C}$ *is a tuple*

$$\mathcal{M} = \langle F, A^*, \Phi \rangle$$

where:

- F *is the* instance facts*, a set of* N_F *ground atoms of the form* $d(o)$ *with descriptor* $d \in \mathcal{D}$ *and object* $o \in \mathcal{T}$ *(e.g.* `d0(o0)`*);*
- A^* *is the* target answer set*, the desired unique stable model that any correct solution* Π *must produce; it must satisfy* $F \subseteq A^*$ *and* $|A^* \setminus F| \geq A_{\min}$*; and*
- Φ *is the* candidate rule schema*, the full set of concrete ASP rules defining the hypothesis space from which* Π *is drawn. Each rule in* Φ *has the form*

  $$d_h(X) \;\texttt{:-}\; p_1(X),\ \ldots,\ p_k(X),\ \texttt{not}\ n_1(X),\ \ldots,\ \texttt{not}\ n_m(X).$$

  where $d_h, p_i, n_j \in \mathcal{D}$*,* X *is a single universally-quantified variable,* $k \leq L^+$ *positive body literals, and* $m \leq L^-$ *negative body literals (bounds from* $\mathcal{C}$*, Definition 1).*

A candidate program Π *is any subset* $\Pi \subseteq \Phi$ *with* $|\Pi| \leq R$*; every rule in* Π *is therefore unary, uses only predicates from* $\mathcal{D}$*, and respects the bounds* L^+ *and* L^-*.* Π *is a* solution *to* $\mathcal{M}$ *if and only if*

$$AS(F \cup \Pi) = \{ A^* \},$$

Meaning the unique answer set of $F \cup \Pi$ *is exactly* A^**.*

The candidate rule schema Φ corresponds directly to the mode bias of the associated ILASP learning task generated for each benchmark.

3.2 Dataset Generation

To instantiate benchmarks across the complexity categories defined above, synthetic data is generated by introducing new possible predicates and terms as needed. To keep the programs sufficiently complex, but manageable by the LLMs, only one stable model is expected and at least one atom must be deduced. The vehicle example above would be represented as shown below.

Listing 2: Benchmark ASP program.

```
d0(o0).
d0(o1).
d1(o1).
d2(X) :- d0(X), not d1(X).
```

This results in the answer set `{ d0(o0), d0(o1), d1(o1), d2(o0) }`, corresponding to the vehicle example answer set.

From each benchmark instance $\mathcal{M}$, a corresponding ILASP task file is generated. The predicates that appear in derived atoms $A^* \setminus F$ are declared as `#modeh` atoms, and all predicates in $\mathcal{D}$ are declared as `#modeb` atoms in both positive and negative forms. Since each benchmark has exactly one answer set, the derived predicates are uniquely determined by A^*, making the `#modeh` declarations both necessary and sufficient. No valid solution can require a head predicate not already present in $A^* \setminus F$. The target answer set A^* is encoded as a single context-dependent `#pos` example with the instance facts F as context. The inclusion set contains the derived atoms $A^* \setminus F$, the exclusion set contains atoms formed by applying each derived predicate to all terms not present in A^*, and the context contains the instance facts. The corresponding ILASP task for the vehicle example above is shown below.

Listing 3: Corresponding ILASP task.

```
#constant(obj, o0).
#constant(obj, o1).

#modeh(d2(var(obj))).

#modeb(1, d0(var(obj))).
#modeb(1, d0(var(obj)), (negative)).
#modeb(1, d1(var(obj))).
#modeb(1, d1(var(obj)), (negative)).
#modeb(1, d2(var(obj))).
#modeb(1, d2(var(obj)), (negative)).

#pos(eg1, {
  d2(o0)
}, {
  d2(o1)
}, {
  d0(o0).
  d0(o1).
  d1(o1).
}).
```

This gives ILASP the same hypothesis space over which LAMP operates. For benchmarks with a sufficiently small hypothesis space, a search space file can be generated by running ILASP in enumeration mode.

3.3 LLM Prompting

LLM prompting has been shown to be effective in writing ASP programs in such areas as logic puzzle solving [19] and robotic task planning [20]. LLM and ASP based hybrid systems have even been shown to be effective in improving natural language understanding [23], common sense reasoning [29] and as collaborative conversations agents [28].

The prompt provided to the LLMs provides the instance facts (ground atoms), such as `descriptor_0(object_0)`, expected ASP output format `predicate_0(X) :- predicate_1(X), [not] predicate_2(X)...`, and the expected stable model, such as `d0(o0), d0(o1), ...`. The output is then parsed into ASP and run with the ASP solver Clingo. This can then be passed back if incorrect. This interaction can be represented as shown below.

Definition 3 (LAMP Interaction Protocol). *A* LAMP Interaction Sequence *for instance $\mathcal{M} = \langle F, A^*, \Phi \rangle$ is a sequence of pairs*

$$\Big\langle\, (\Pi_0, \delta_0),\ (\Pi_1, \delta_1),\ \ldots,\ (\Pi_n, \delta_n) \,\Big\rangle$$

where each Π_i is a candidate program generated by the LLM given F, A^*, and all prior feedback, and δ_i is the feedback signal *produced by running $F \cup \Pi_i$ through Clingo:*

$$\delta_i \;=\; \begin{cases} \texttt{correct} & \text{if } AS(F \cup \Pi_i) = \{A^*\}, \\ \texttt{unsatisfiable} & \text{if } AS(F \cup \Pi_i) = \emptyset, \\ \big\langle\, AS(F \cup \Pi_i),\ A^* \,\big\rangle & \text{otherwise.} \end{cases}$$

The sequence terminates when $\delta_n = \texttt{correct}$ or a maximum iteration count is reached. The final output Π_n is accepted as the solution if it satisfies $\mathcal{M}$ (Definition 2).

As future work the prompt may be able to be optimized by adding examples or using DSPy [17] which tests many variations of the prompt with different examples to get the best result. This was demonstrated for spatial reasoning as a chain of thought reasoning system [26]. Another option would be to fine tune the LLM specifically for ASP code generation as was done with LLASP [6].

3.4 Evaluation

LAMP and ILASP systems are evaluated by correctness of answer set, time to solve and optimality of ASP programs. The ASP programs generated are evaluated after being run via Clingo. The correctness, number of rules, complexity and how close it is to the expected stable model can then be assessed. This is similar to the process LLM-ARC employs as an actor-critic method, where the LLM Actor generates ASP, while the automated reasoning critic evaluates the code [15]. Other systems use ASP as a scaffolding for robust reasoning [16].

Beyond correctness, the systems are evaluated on whether the LLM has produced a more concise encoding than the original synthetic program, formalized as follows.

Definition 4 (Program Conciseness). *Given a solution* Π *to instance* $\mathcal{M}$*, its* literal count *is*

$$L(\Pi) \;=\; \sum_{r \in \Pi} \big(pos(r) + neg(r)\big),$$

where $pos(r)$ *and* $neg(r)$ *are the sets of distinct positive and negative body literals of rule* r*. A solution* Π *is* more concise *than* Π' *if* $L(\Pi) < L(\Pi')$.

Many other notions of optimality are applicable to ASP programs, including body literal and rule count, stratification, tightness, total grounded rules, and ASP solving time. This work focuses on literal count as a direct measure of complexity with stratification reported separately.

4 Experiments

Experiments where conducted on the synthetic data with varying complexities as specified in the section *ASP, Complexity and Datasets*. A variety of online and local open weight LLMs where selected for testing to get a broad view of capabilities.

4.1 Experimental Setup

A total of 120 synthetic ASP programs were created with varying features for testing. The LLMs chosen included *gpt-5-mini*, which was run via OpenAI's API platform, as well as open weight models *gpt-oss:20b* and *qwen3-coder:30b* which were run locally using Ollama. All ASP solving and analysis was done using Clingo. ILASP was run on each benchmark with a timeout of 420 seconds. Benchmarks not solved within this limit were recorded as failures. The selected parameters used for complexity classes are listed in table 2. All combinations were used except limited where positive and negative literals must sum to less than overall literals.

4.2 Results and Discussion

Overall results are shown in table 2. Both *gpt-5-mini* and *gpt-oss:20b* perform very well, deducing the correct ASP rules to get the expected stable model in almost all cases. LAMP using *qwen3-coder:30b* performed significantly worse, reaching only 45.0% final accuracy (54/120), with 30.8% correct on the first attempt. Although *qwen3-coder:30b* is tuned for programming, it may not have been tuned for ASP, resulting in poorer performance. Of the

Table 1: Parameters associated with Definition 1

Parameter	**Variations**
Descriptors	[10, 6]
Objects	[10, 6]
Facts	[9, 6]
Rules	[11]
Overall Literals	[3, 2]
Positive Literals	[2, 1, 0]
Negative Literals	[1]

cases the correct stable model was produced the rules never exactly matched the ones from the synthetic data.

ILASP achieved 97.5% accuracy (117/120) of which all three failures were timeouts, not reasoning errors, occurring on benchmarks with high negation counts and large numbers of derived atoms. LLMs in LAMP *gpt-5-mini* and *gpt-oss:20b* solved 3 and 1 benchmarks respectively of which ILASP timed out demonstrating that LLM-guided search can succeed in some instances where ILP search cannot.

The iterative feedback loop contributes meaningfully for all models. First attempt accuracy was 94.2% for *gpt-5-mini* (113/120), 90.0% for *gpt-oss:20b* (108/120), and 30.8% for *qwen3-coder:30b* (37/120). Subsequent Clingo verified feedback iterations recovered 7 additional cases for *gpt-5-mini*, 7 for *gpt-oss:20b*, and 17 for *qwen3-coder:30b*.

Table 2: Comparison of all systems including ILASP baseline. First attempt is the accuracy on the initial generation; final accuracy is after up to 3 feedback iterations. ILASP failures are all timeouts (420 s limit); LLM failures are unresolved after all iterations.

System	First attempt	Final	Failures	Failure type
ILASP	—	117/120 (97.5%)	3	Timeout only
gpt-5-mini	113/120 (94.2%)	120/120 (100%)	0	—
gpt-oss:20b	108/120 (90.0%)	115/120 (95.8%)	5	—
qwen3-coder:30b	37/120 (30.8%)	54/120 (45.0%)	66	—

Table 3 details the three benchmarks on which ILASP timed out (benchmark indices 7, 27, and 67). All share the structural profile of maximum facts (9), elevated negative literal counts, and approximately three times the average number of derived atoms. *gpt-5-mini* solved all three, *gpt-oss:20b* solved benchmark 7 but failed on 27 and 67 while *qwen3-coder:30b* failed all three.

Table 3: Outcomes for the 3 benchmarks where ILASP timed out (420 s limit). ✓= correct stable model produced; × = failed.

Benchmark	gpt-5-mini	gpt-oss:20b	qwen3-coder:30b	ILASP
7	✓	✓	×	Timed out
27	✓	×	×	Timed out
67	✓	×	×	Timed out

Figure 2 shows the breakdown of benchmark outcomes per LLM across the three categories of correct on the first attempt (green), correct only after Clingo verified feedback (gold), and failed across all iterations (red).

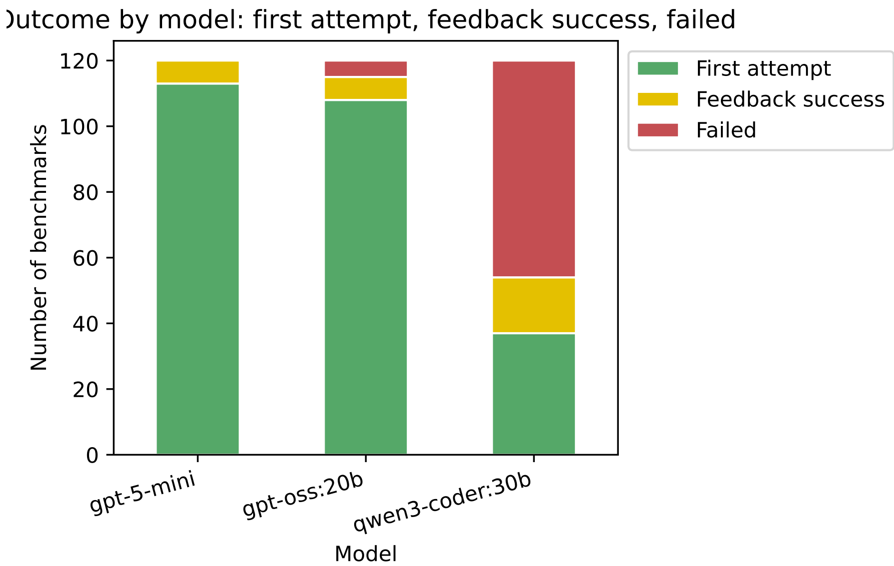

Figure 2: Stacked bar chart of benchmark outcomes per model: first-attempt success (green), feedback success (gold), and failed (red). Produced from `stats.ipynb` cell 2.

Figure 3 shows the per-benchmark wall-clock time distribution for each system as a box plot. *gpt-5-mini* is the fastest LLM with a median of 18.0 and a maximum of 137.0, reflecting the low latency of the API-hosted model. *gpt-oss:20b* is slowest with median 50.8 and max 369.6 seconds due to the larger locally-run model size. ILASP has a highly bimodal distribution. The median is only 0.66 seconds as most benchmarks are trivially solved by exhaustive search, but three benchmarks hit the 420.0 second ceiling, pulling the maximum to that value. *qwen3-coder:30b* shows a near uniform distribution with a median of 40.3 and a maximum near the 420.0 timeout at 417.8 seconds, consistent with the model spending maximum iteration budget on hard cases it ultimately fails to solve. Table 4 gives the aggregate statistics.

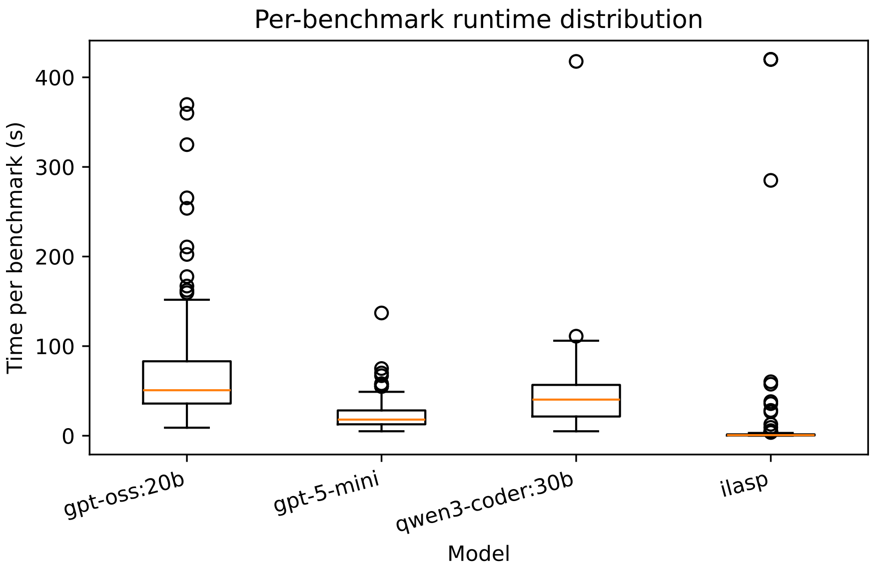

Figure 3: Per-benchmark wall-clock time distribution per system (seconds). Boxes show the interquartile range; whiskers extend to the full range. ILASP failures are capped at the 420 s timeout. Produced from `stats.ipynb` cell 5.

Table 4: Wall-clock time per system (seconds). LLM times accumulate all feedback iterations; ILASP timeouts are recorded as 420 s. Produced from `stats.ipynb` cell 5.

Model	Total (s)	Mean (s)	Median (s)	Max (s)
gpt-5-mini	2837.00	23.64	18.00	137.00
gpt-oss:20b	8579.97	71.50	50.78	369.64
qwen3-coder:30b	5319.65	44.33	40.32	417.77
ILASP	1917.36	15.98	0.66	420.00

Table 5.

Figures 4, 5 and 6 compare the total literal count $L(\Pi)$ (Definition 4) of the original ASP programs against that of the LLM generated rules for each LLM tested. The dot size indicates the number of examples with left for when the LLM got the correct stable model and right for incorrect. Figure 5 shows `gpt-5-mini` is generally able to greatly reduce the number of literals needed, in some case reducing number of literals by 6, but in some cases increas-

LLM	Correct	Number Correct Stratified
Original	—	49
ILASP	117/120	117
gpt-5-mini	120/120	119
gpt-oss:20b	115/120	114
qwen3-coder:30b	54/120	54

Table 5: LLM with Answer Set Correct and Incorrect Results, Rules Matching Indicates number of LLM Generated Programs that Match the Original.

ing them by 2. LLM `qwen3-gpt_oss_literals:20b` shown in figure 5 had similar results generally needing even fewer literals, while `qwen3_literals` shown in figure 6 had mixed results. Overall, the results corroborate with other benchmarking studies on the abilities of LLMs to generate ASP [24].

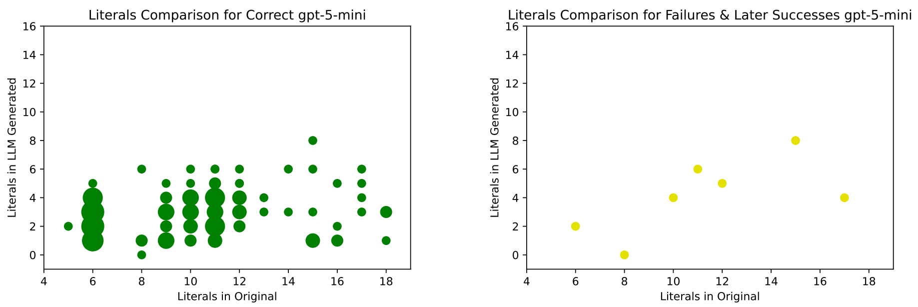

Figure 4: Gpt-5-mini input literals to output literals for correct answer set response.

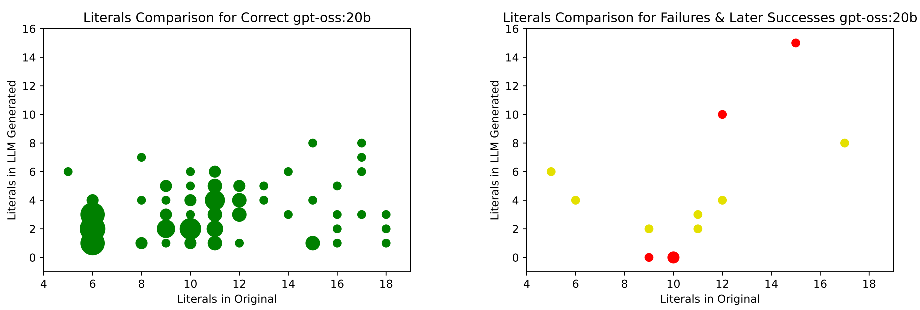

Figure 5: Gpt-oss:20b input literals to output literals for correct answer set response.

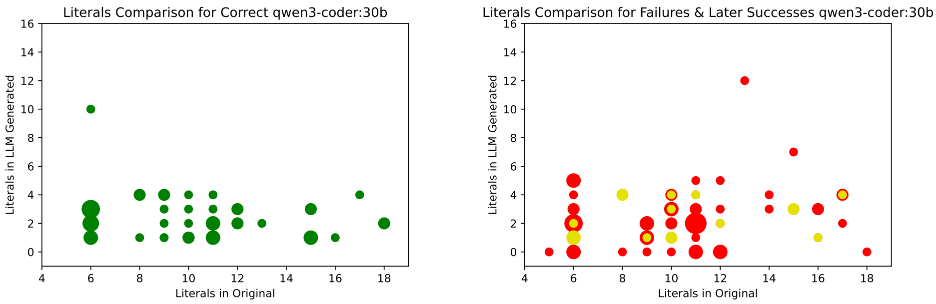

Figure 6: Qwen3-coder input literals to output literals for correct answer set response.

4.3 Feature Importance

Based on whether the LLM is able to get the correct stable model or not it is possible to deduced which ASP rule features are most difficult for LLMs to deduce thereby indicating higher complexity. To do this, the features of the original ASP program and the LLM deduced ASP program such as number of rules, number of new atoms inferred etc. are used to predict the likelihood of getting the correct stable model using the tree based machine learning algorithm XGBoost.

From the XGBoost model the importance value of each feature can be calculated. There are multiple ways to get importance, such as gain, or average improvement in the loss function from splits using that feature or cover, the average number of samples affected by splits using that feature. For this analysis weight is used, the number of times a feature is used to split across all trees [21].

The results can be seen in figure 7. The most important feature according to this analysis is *atom_diff_llm*, or the number of new atoms infered by the LLMs program. This makes sense as the LLMs ASP not inferring atoms is a good indicator it won't arrive at the expected stable model. The next most important feature is *total_neg_literals_llm*, indicating that having a lot of negative literals adds complexity.

A similar method that is somewhat more robust involves deriving the features SHAP values. Each feature's SHAP value is the average change in prediction caused by that feature taken over all possible ways that feature could have appeared in the model [21]. SHAP values shown in figure 8 show similar results to figure 7.

An ASP program is *stratified* if no predicate depends negatively on itself (directly or transitively). Stratification is a structural property that simplifies semantics and is often correlated with lower solver complexity. Since the LAMP benchmarks include programs with varying negation depths, it is informative to assess whether LLM-generated programs preserve this property.

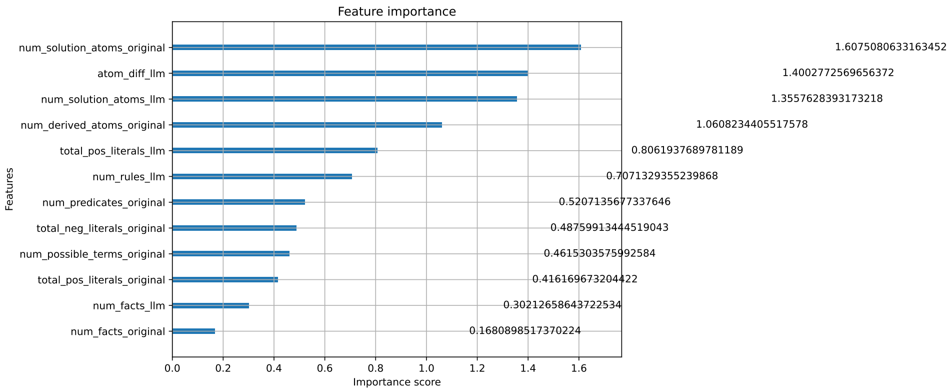

Figure 7: XGBoost Feature Importance

Analysis of the output programs (computed via the stratification routine in the LAMP implementation, which uses Kosaraju's algorithm to detect strongly connected components in the predicate dependency graph) shows that *gpt-5-mini* and *gpt-oss:20b* preserve stratification in virtually all correct outputs. *qwen3-coder:30b* shows more frequent violations, often introducing unnecessary cyclic negative dependencies. Stratification depth is positively correlated with solver time on correct benchmarks, confirming it as a proxy for structural complexity. Preservation of stratification thus provides an additional structural correctness metric beyond answer-set equality.

5 Related Work

5.1 Inductive ASP Learning and LAS

Recent work has explored the integration of LLMs and ASP with several approaches studying how LLMs can learn from or reason over ASP directly. Law et al. [?] introduced ILASP, the state-of-the-art system for Learning from Answer Sets, which exhaustively searches a mode-bias-defined hypothesis space for optimal inductive solutions and is used as the classical ILP baseline in this work. Borroto et al. (2025) [4] investigates question answering and inductive learning from answer sets, introducing the LLM2LAS system, which uses LLMs to construct ILASP learning tasks from natural language stories and then runs ILASP to induce the required commonsense rules. A key distinction from LAMP is that LLM2LAS uses LLMs to *build the ILP task* and then defers to ILASP for search, whereas LAMP uses LLMs to *directly generate* the hypothesis, bypassing exhaustive search entirely. Declarative knowledge distillation techniques have also been proposed

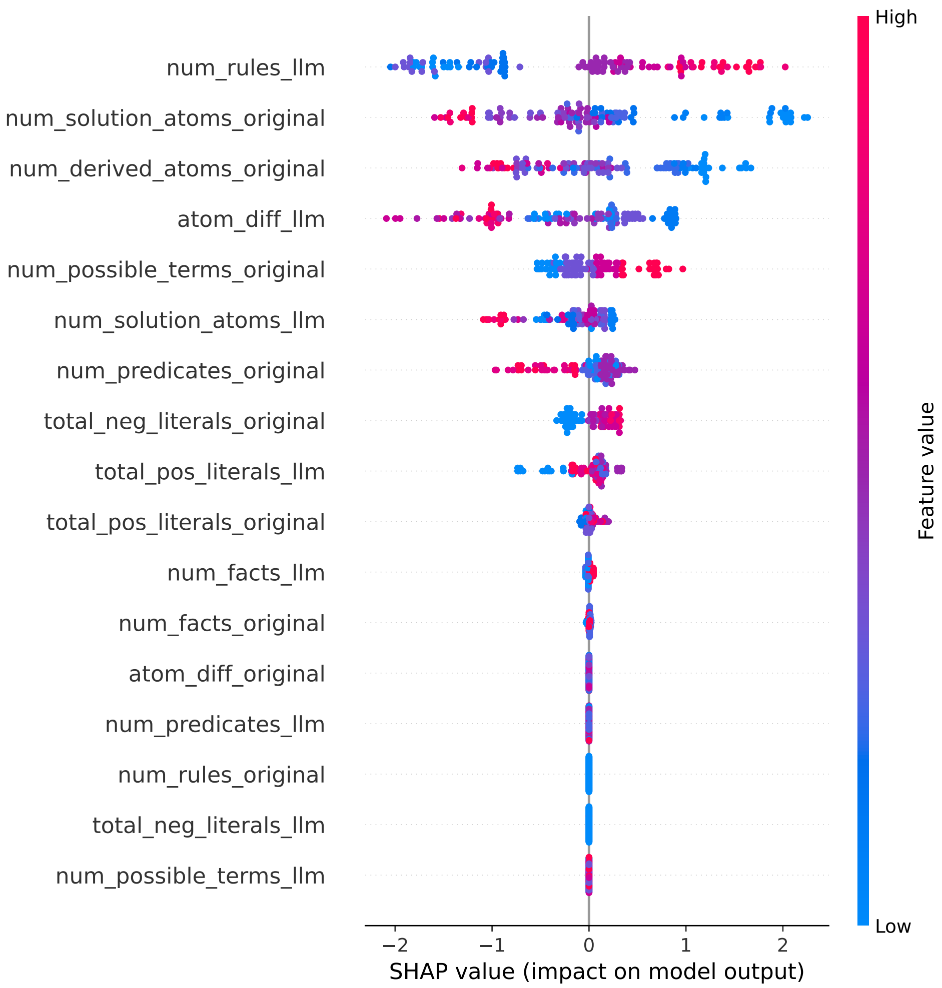

Figure 8: SHAP Feature Importance

to transfer structured reasoning from LLMs into symbolic representations by Either et al. (2024) [8]. Similarly, Borroto et al. (2024) [5] demonstrates automatic composition of ASP programs from natural language specifications, focusing on translating high level descriptions into executable ASP encodings.

5.2 Expressivity-Graded Evaluation

Gandarela et al. [10] introduced the RuDaS benchmark framework and a systematic methodology for evaluating LLM-based theory induction graded by rule expressivity—Chain, RDG, DRDG, and Mixed categories (see also Section 2.2). A key finding from that work is that LLMs are more limited by their capacity to track long predicate-dependency chains (Chain category) than by disjunctive complexity (DRDG); this is directly relevant to future LAMP extensions beyond the current Chain/RDG fragment.

Beyond ASP specific research, logic programming has been used as a tool to evaluate and scaffold LLM reasoning. Hybrid GOFAI LLM systems show how expert systems can be rebuilt using LLM generated logic rules by Garrido et al. (2025) [11], while Xu et al. (2025) [27] shows programming based test evaluators systematically assess LLM reasoning reliability . These works support the use of formal solvers, as in LAMP, to ground and validate LLM outputs.

5.3 Planning and Neuro-Symbolic Pipelines

Planning provides another closely related line of research. Logical chain of thought tuning for symbolic planning by Verma et al. (2025) [25] and LLM symbolic planning pipelines without expert input by Huang et al. (2024) [13] highlight the benefits of keeping solvers such as ASP planners in the loop. Established ASP planning benchmarks, including Plasp developed by Dimopoulos et al. [7], offer realistic domains that could extend LAMP beyond product configuration. Neuro symbolic refinement frameworks such as PEIRCE further demonstrate how LLMs and formal reasoning systems can iteratively improve each other [22] by Quan et al. (2025).

6 Future Work

Several directions for future work follow naturally. First, iterative prompting could be improved by more explicit chain of thought reasoning, potentially evaluating candidate rules across multiple instance fact sets per iteration, with Clingo acting as a tool within an LLM based agent loop. Prompt optimization frameworks such as DSPy could be used to automatically refine prompts and feedback strategies. Second, the synthetic datasets could be extended to richer ASP constructs, including higher arity predicates, choice rules, disjunction, and head disjunction, as well as more complex dependency structures such as those proposed by Gandarela et al. (2025) regarding logic program expressivity [10].

Third, evaluation could move beyond synthetic configuration tasks to existing ASP datasets, including OOASP models, real world ASP encodings, planning benchmarks such as Plasp and PDDL to ASP translations, and domains like rail scheduling. Fourth, deeper natural language interfaces to ASP could be explored, covering bidirectional translation between text and ground facts, rules, stable models, and multiple natural languages. Finally, optimization remains a promising avenue, including other ways of transforming ASP into more efficient ASP, compiling ASP fragments to lower level code, such as C, and tighter integration with solver level optimizations such as those seen in the Non Ground Optimizer (NGO), enabling LLMs to reason not only about correctness but also about grounding and solving performance.

7 Conclusion

This work proposed a structured LLM to ASP Modeling Protocol (LAMP) designed to support the deduction and optimization of ASP rules from instance facts and expected stable models. The results show this as effective for guiding LLMs toward correct and often optimized ASP encodings. By constraining the modeling task and providing iterative feedback via an ASP solver, this system reduces the modeling burden associated with ASP while preserving correctness.

The tested models exhibited clear performance differences, with ~~*gpt-5-mini* and *gpt-oss:20b* achieving solid stable model reconstruction, however *qwen3-coder:30b* struggled despite its programming focus.~~*gpt-5-mini* achieving 100% final accuracy (94.2% on first attempt) and *gpt-oss:20b* achieving 95.8% (90%), while *qwen3-coder:30b* reached only 45% (30.8% first attempt) despite its programming focus. These results highlight that strong general code generation ability does not necessarily translate to proficiency in logic programming, underscoring the value of ASP specific evaluation frameworks. Analysis of complexity features and feature importance, shows that the presence of negative literals and the ability to correctly infer new atoms are the strongest predictors of LLM success.

The ILASP baseline achieved 97.5% (117/120), with all three failures being search-space timeouts rather than reasoning errors. Crucially, both *gpt-5-mini* and *gpt-oss:20b* solved all three benchmarks on which ILASP timed out. This demonstrates that LLMs do not merely replicate classical ILP search—they navigate the hypothesis space in a fundamentally different way and can succeed precisely where exhaustive search cannot. The iterative feedback loop further contributed recovery of 7, 7, and 17 cases for the three models respectively, confirming its value as a component of the LAMP pipeline.

Beyond assisting users in ASP modeling, this protocol provides a system for probing LLM reasoning and logic programming capabilities. Future work includes extending to richer ASP constructs such as disjunction and choice rules, incorporating solver-level performance metrics, and exploring fine-tuned or neuro-symbolic hybrids to further improve robustness and scalability.

References

[1] Amendola, G., Ricca, F., Truszczynski, M.: Beyond np: Quantifying over answer sets (2019).

https://doi.org/https://doi.org/10.1017/S1471068419000140, https://arxiv.org/abs/1907.09559

[2] Bichler, M., Morak, M., Woltran, S.: The power of non-ground rules in answer set programming (2016), https://arxiv.org/abs/1608.01856

[3] Bogaerts, B., Charalambidis, A., Chatziagapis, G., Kostopoulos, B., Pollaci, S., Rondogiannis, P.: The stable model semantics for higher-order logic programming (2024). https://doi.org/https://doi.org/10.1017/S1471068424000231, https://arxiv.org/abs/2408.10563

[4] Borroto, M., Gallagher, K., Ielo, A., Kareem, I., Ricca, F., Russo, A.: Question answering with llms and learning from answer sets (2025), https://arxiv.org/abs/2509.16590

[5] Borroto, M., Kareem, I., Ricca, F.: Towards automatic composition of asp programs from natural language specifications (2024), https://arxiv.org/abs/2403.04541

[6] Coppolillo, E., Calimeri, F., Manco, G., Perri, S., Ricca, F.: Llasp: Fine-tuning large language models for answer set programming (2024), https://arxiv.org/abs/2407.18723

[7] Dimopoulos, Y., Gebser, M., Lühne, P., Romero, J., Schaub, T.: plasp 3: Towards effective asp planning (2018). https://doi.org/https://doi.org/10.1017/S1471068418000583, https://arxiv.org/abs/1812.04491

[8] Eiter, T., Hadl, J., Higuera, N., Oetsch, J.: Declarative knowledge distillation from large language models for visual question answering datasets (2024), https://arxiv.org/abs/2410.09428

[9] Falkner, A., Ryabokon, A., Schenner, G., Shchekotykhin, K.: Ooasp: Connecting object-oriented and logic programming (2015), https://arxiv.org/abs/1508.03032

[10] Gandarela, J.P., Carvalho, D.S., Freitas, A.: Inductive learning of logical theories with llms: An expressivity-graded analysis (2025), https://arxiv.org/abs/2408.16779

[11] Garrido-Merchán, E.C., Puente, C.: Gofai meets generative ai: Development of expert systems by means of large language models (2026), https://arxiv.org/abs/2507.13550

[12] Gebser, M., Kaminski, R., Kaufmann, B., Schaub, T.: Clingo = asp + control: Preliminary report (2014), https://arxiv.org/abs/1405.3694

[13] Huang, S., Lipovetzky, N., Cohn, T.: Planning in the dark: Llm-symbolic planning pipeline without experts (2024), https://arxiv.org/abs/2409.15915

[14] Ishay, A., Yang, Z., Lee, J.: Leveraging large language models to generate answer set programs (2023), https://arxiv.org/abs/2307.07699

[15] Kalyanpur, A., Saravanakumar, K.K., Barres, V., Chu-Carroll, J., Melville, D., Ferrucci, D.: Llm-arc: Enhancing llms with an automated reasoning critic (2024), https://arxiv.org/abs/2406.17663

[16] Kaur, N., McPheat, L., Russo, A., Cohn, A.G., Madhyastha, P.: An empirical study of conformal prediction in llm with asp scaffolds for robust reasoning (2025), https://arxiv.org/abs/2503.05439

[17] Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Vardhamanan, S., Haq, S., Sharma, A., Joshi, T.T., Moazam, H., Miller, H., Zaharia, M., Potts, C.: Dspy: Compiling declarative language model calls into self-improving pipelines (2023), https://arxiv.org/abs/2310.03714

[18] Law, M., Russo, A., Broda, K.: The ilasp system for inductive learning of answer set programs (2020), https://arxiv.org/abs/2005.00904

[19] Li, N., Liu, P., Liu, Z., Dai, T., Jiang, Y., Xia, S.T.: Logic-of-thought: Empowering large language models with logic programs for solving puzzles in natural language (2025), https://arxiv.org/abs/2505.16114

[20] Lin, X., Wu, Y., Yang, H., Zhang, Y., Zhang, Y., Ji, J.: Clmasp: Coupling large language models with answer set programming for robotic task planning (2024), https://arxiv.org/abs/2406.03367

[21] Lundberg, S.M., Erion, G.G., Lee, S.I.: Consistent individualized feature attribution for tree ensembles (2019), https://arxiv.org/abs/1802.03888

[22] Quan, X., Valentino, M., Carvalho, D.S., Dalal, D., Freitas, A.: Peirce: Unifying material and formal reasoning via llm-driven neuro-symbolic refinement (2025), https://arxiv.org/abs/2504.04110

[23] Rajasekharan, A., Zeng, Y., Padalkar, P., Gupta, G.: Reliable natural language understanding with large language models and answer set programming (2023). https://doi.org/https://doi.org/10.4204/EPTCS.385.27, https://arxiv.org/abs/2302.03780

[24] Ren, L., Xiao, G., Qi, G., Geng, Y., Xue, H.: Can llms solve asp problems? insights from a benchmarking study (extended version) (2025), https://arxiv.org/abs/2507.19749

[25] Verma, P., La, N., Favier, A., Mishra, S., Shah, J.A.: Teaching llms to plan: Logical chain-of-thought instruction tuning for symbolic planning (2025), https://arxiv.org/abs/2509.13351

[26] Wang, R., Sun, K., Kuhn, J.: Dspy-based neural-symbolic pipeline to enhance spatial reasoning in llms (2024), https://arxiv.org/abs/2411.18564

[27] Xu, Z., Ding, J., Lou, Y., Zhang, K., Gong, D., Li, Y.: Socrates or smartypants: Testing logic reasoning capabilities of large language models with logic programming-based test oracles (2025), https://arxiv.org/abs/2504.12312

[28] Zeng, Y., Gupta, G.: Reliable collaborative conversational agent system based on llms and answer set programming (2025), https://arxiv.org/abs/2505.06438

[29] Zeng, Y., Rajashekharan, A., Basu, K., Wang, H., Arias, J., Gupta, G.: A reliable common-sense reasoning socialbot built using llms and goal-directed asp (2024). https://doi.org/https://doi.org/10.1017/S147106842400022X, https://arxiv.org/abs/2407.18498